APP DE GERENCIAMENTO PARA PET SHOP

PetCare

Cadastro de animais, serviços e histórico de atendimentos em um só lugar.

React Native · Expo · Node.js · Express · MySQL

Ana Sprenger



VISÃO GERAL

O que o PetCare resolve

Um só lugar para cadastrar os animais, registrar os serviços e acompanhar o histórico de cada pet.

Animais

Nome, espécie, raça, idade, tutor, telefone e observações.

Serviços

Animal atendido, tipo, data, valor, descrição e status.

Painel

Totais por status, faturamento e próximos agendamentos.

FUNCIONALIDADES

CRUD completo das duas entidades

Animais

- Listagem com busca por nome, raça ou tutor
- Cadastro com validação dos campos
- Detalhes com o histórico de serviços
- Edição dos dados
- Exclusão com confirmação

Serviços

- Listagem com filtro por status
- Cadastro ligado a um animal
- Detalhes do serviço e do tutor
- Edição dos dados
- Exclusão com confirmação

Tipos de serviço Banho | Tosa | Consulta veterinária | Vacinação | Hospedagem | Outro

Status Agendado | Em andamento | Concluído

ARQUITETURA

Três camadas que conversam entre si

FRONTEND

App

- React Native + Expo
- React Navigation: abas e telas empilhadas
- Roda no celular ou no navegador

fetch + JSON



BACKEND

API REST

- Node.js + Express
- Rotas de animais, serviços e resumo
- Valida os dados recebidos

SQL com ?



BANCO DE DADOS

MySQL

- Banco petshop_db
- Tabelas animais e servicos
- Acesso pelo driver mysql2

O app nunca acessa o banco direto: toda leitura e gravação passa pela API.

BANCO DE DADOS

petshop_db: duas tabelas relacionadas

animais

Coluna	Tipo
id	INT (chave primária)
nome	VARCHAR(100)
especie	VARCHAR(50)
raca	VARCHAR(100)
idade	INT
tutor	VARCHAR(100)
telefone	VARCHAR(20)
observacoes (nova)	VARCHAR(500)

1 : N

um animal tem vários serviços

→

ON DELETE CASCADE

excluir o animal remove os serviços dele

servicos

Coluna	Tipo
id	INT (chave primária)
animal_id	INT (chave estrangeira)
tipo	VARCHAR(100)
data	DATE
valor	DECIMAL(10,2)
status	VARCHAR(30)
observacoes	TEXT

API REST

Rotas do backend

Método	Rota	O que faz
GET	/api/resumo	Totais do painel e próximos agendamentos
GET	/api/animais	Lista os animais com o total de serviços
GET	/api/animais/:id	Detalhes do animal e histórico de serviços
POST	/api/animais	Cadastra um animal
PUT	/api/animais/:id	Atualiza um animal
DELETE	/api/animais/:id	Exclui o animal e os serviços dele
GET	/api/servicos?status=	Lista os serviços, com filtro opcional
GET	/api/servicos/:id	Detalhes do serviço com o animal
POST	/api/servicos	Cadastra um serviço
PUT	/api/servicos/:id	Atualiza um serviço
DELETE	/api/servicos/:id	Exclui um serviço

DESIGN

Visual inspirado na Apple

- Tons de rosa suave, branco e cinza-claro
- Cartões arredondados e listas agrupadas, como no iOS
- Tipografia do sistema, com títulos grandes
- Navegação por abas: Início, Animais e Serviços
- Estados visuais: carregando, vazio, erro e confirmação

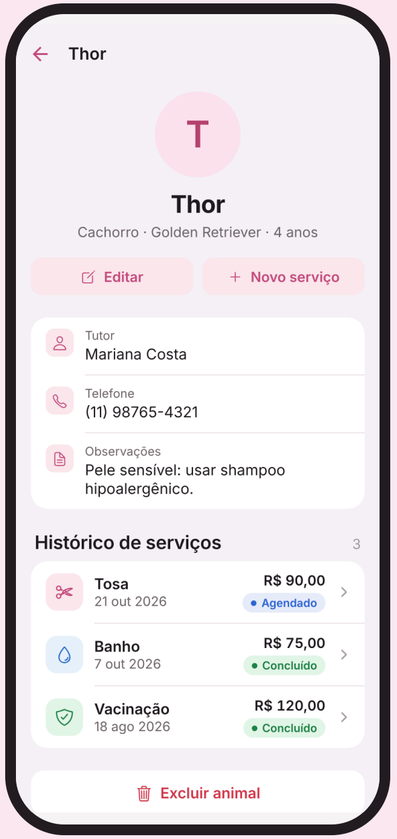
Rosa
#C74B7D

Rosa suave
#FBE7EF

Cinza-claro
#F6F3F5

Branco
#FFFFFF

Texto
#221C20



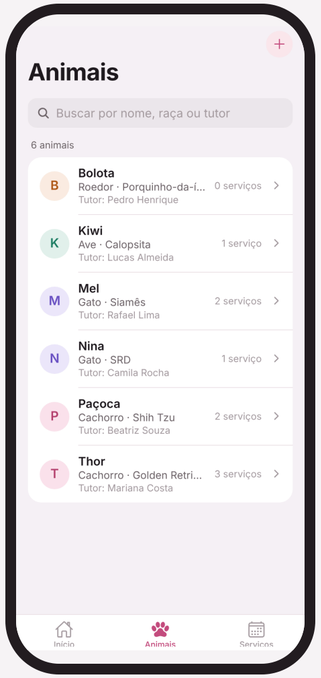
TELAS

As três abas do app



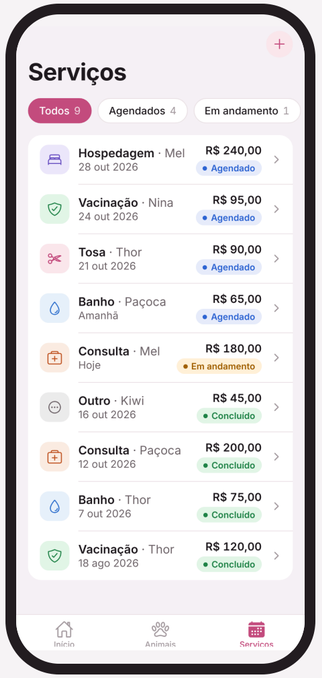
Início

Painel de resumo e próximos agendamentos



Animais

Lista com busca e total de serviços de cada pet

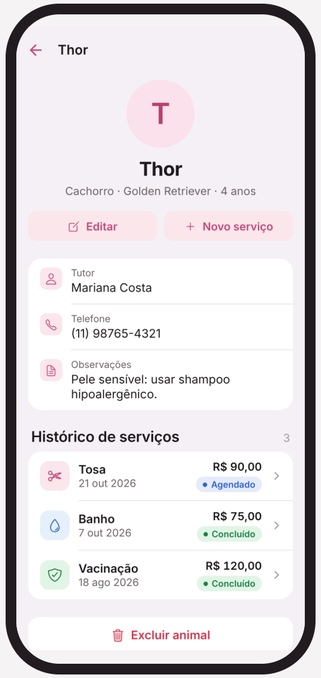


Serviços

Filtro por status, valor e data de cada serviço

TELAS

Detalhes e formulários



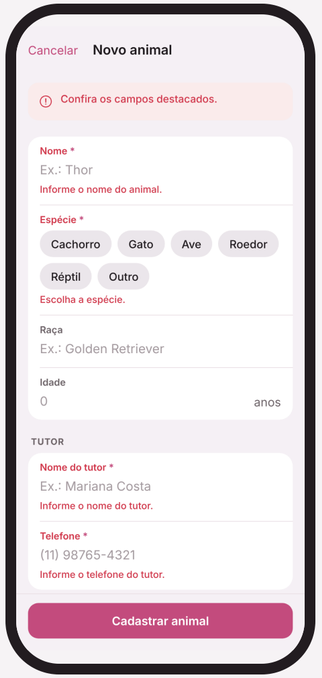
Detalhe do animal

Dados do tutor e histórico de serviços



Novo serviço

Animal, tipo, data, valor e status



Validação

Campos obrigatórios destacados com a mensagem

CÓDIGO

Como o app chama a API

Trecho simplificado de mobile/src/services/api.js

```
async function request(path, { method = 'GET', body } = {}) {
  const controller = new AbortController();
  setTimeout(() => controller.abort(), 10000); // limite de 10 s
  const response = await fetch(API_URL + path, {
    method, signal: controller.signal,
    headers: { 'Content-Type': 'application/json' },
    body: body && JSON.stringify(body),
  }).catch(() => {
    throw new ApiError('Não foi possível conectar ao servidor.');
  });
  const data = await response.json();
  if (!response.ok) throw new ApiError(data.erro, response.status);
  return data;
}
```

Tempo limite

Sem resposta em 10 segundos, a requisição é cancelada.

Erros com mensagem

A API responde com o erro e o app mostra a mensagem na tela.

Sem conexão

O app avisa e oferece o botão “Tentar novamente”.

CÓDIGO

Exclusão sem registros órfãos

Trecho simplificado de backend/src/routes/animais.js

```
router.delete('/:id', async (req, res) => {
  const conexao = await pool.getConnection();
  try {
    await conexao.beginTransaction();
    await conexao.execute(
      'DELETE FROM servicos WHERE animal_id = ?', [id]);
    await conexao.execute('DELETE FROM animais WHERE id = ?', [id]);
    await conexao.commit(); // tudo ou nada
  } catch (erro) {
    await conexao.rollback(); // desfaz se algo falhar
    throw erro;
  } finally { conexao.release(); }
});
```

Transação

Os serviços e o animal saem juntos. Se algo falhar, nada muda.

Consulta parametrizada

O ? recebe o valor separado do SQL, o que evita SQL injection.

Garantia no banco

A chave estrangeira com ON DELETE CASCADE faz o mesmo no MySQL.

QUALIDADE

Validação e tratamento de erros

Campos obrigatórios

Verificados no app e de novo na API, que responde 400 com o campo e a mensagem.

Consultas parametrizadas

Os valores entram no lugar do ?, nunca colados no texto do SQL.

Confirmação ao excluir

O app pergunta antes de apagar um animal ou um serviço.

Respostas HTTP tratadas

404 para registro não encontrado e 503 quando o banco está fora do ar.

Sem registros órfãos

Transação na API e ON DELETE CASCADE no banco.

Estados de tela

Carregando, lista vazia, erro com “Tentar novamente” e aviso de sucesso.

DEMONSTRAÇÃO

Como executar o projeto

1

Preparar o banco (uma vez)

```
mysql -u root -p < database/atualizar_petshop_db.sql
```

2

Ligar a API

```
cd backend && npm start
```

Responde em http://localhost:3000/api

3

Abrir o app no navegador

```
cd mobile && npx expo start --web
```

Abre em http://localhost:8081

PETCARE

Perguntas?

React Native · Node.js · Express · MySQL